

Copper-Policy: Focus on the Representation for Robust Robot Manipulation

Zexin Feng¹ Yixu Feng² Lingyu Xiao¹ Shang Su^{3,4} Kexin Zheng¹
 Chang Xu² Mengkai Shi⁴ Shuo Feng³ Xintao Yan^{1,✉}
¹The University of Hong Kong ²The University of Sydney
³Tsinghua University ⁴DenseAI Inc.

Compact · Intention-Driven · Action-Decodable

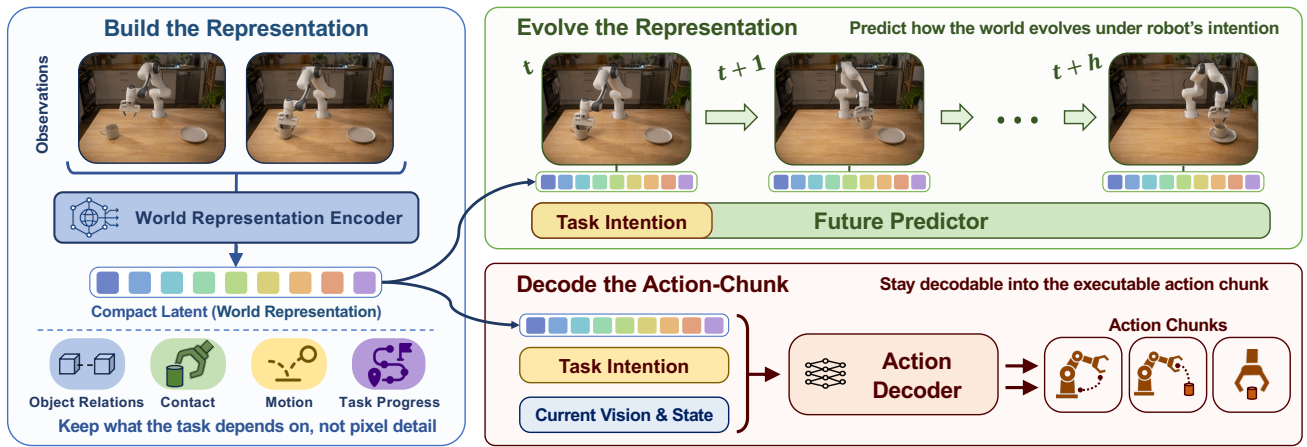


Figure 1. **Overview of our formulation.** A World representation for control should be compact,¹ evolve under task intention,² and remain decodable into actions.

Abstract

World Action Models (WAMs) acquire behavioral priors by modeling future scene evolution, but predicting detailed futures in pixel or latent space incurs substantial cost. Recent evidence that co-training gains persist without test-time generation raises a question: what must a WAM learn to improve control? We introduce **Copper-Policy**, which learns a compact World representation with the policy rather than relying on a predefined target space. Through **temporal joint-embedding prediction**, it predicts future observation embeddings conditioned on task intention without reconstructing pixels. This prediction and action decoding shape the representation jointly, while the policy retains access to current-frame spatial detail for execution. Representation analyses show that the learned features better separate task-driven change from perturbations and provide complementary information for control. Compact prediction

targets reduce training tokens per sample, enabling a **2B-parameter model** trained in **9.67 hours on 8× RTX 5090 GPUs**, about $6\times$ faster than Fast-WAM. Copper-Policy leads all compared methods without embodied pretraining on RoboTwin and outperforms several embodied-pretrained VLAs on LIBERO-Plus (80.85%). On three real-robot tasks, its 96.3% average is comparable to $\pi_{0.5}$. Together, these results support strong control performance and efficient training.

1. Introduction

Robotic research has long pursued autonomous systems equipped with human-like cognition to enable daily physical interaction. This pursuit has been accelerated by recent advances in the concept of **world modeling**, which offers a shared substrate for building decision-oriented cognition that can both *comprehend* and *predict what will happen*, specialized for downstream embodiment control. One

✉Corresponding author.

prominent backbone is the Vision-Language-Action (VLA) model, which capitalizes on the semantic capabilities of pretrained Vision-Language Models (VLMs) to establish a direct mapping from visual observations and language instructions to executable robot actions [21, 37, 43, 52]. In contrast, the emerging line, World Action Models (WAMs), go further by modeling future scene evolution through visual dynamics [25, 46], explicitly linking *what the world should become* to *how to achieve it*. Yet despite their effectiveness, generative WAMs couple action with explicit video (or latent video) generation, leading to substantial training burden and inference overhead.

This limitation necessitates a reassessment: *what aspect of dynamics learning in WAM actually drives better control?* Fast-WAM [46] demonstrates that **video co-training** remains beneficial even without test-time future generation, while more recent studies indicate that the learned embeddings that encode informative future representations could actively shape policy decisions [33, 39, 53]. The value of WAMs thus appears to lie less in the futures they generate than in the representations their training shapes, motivating a **latent-prediction paradigm** that predicts future states directly in representation space. Existing designs largely fall into two categories. One line uses a pretrained Joint-Embedding Predictive Architecture (JEPA) space either as additional policy input or as fixed prediction targets [27, 40]. The other constructs a predictive latent under fixed constraints: LiLa-WAM [41] compresses current features with learned queries but decodes its predicted future into frozen DINOv3 features, while StageWAM [31] freezes V-JEPA2 on both sides and then freezes its predictor before policy learning. In both cases, the **predictive target space does not co-evolve with the policy** that ultimately consumes it.

To instantiate this perspective, we introduce **Copper-Policy**, which treats predictive representation learning as part of the policy itself. Rather than requiring the model to reconstruct detailed futures, Copper-Policy distills task-relevant scene evolution into a compact latent representation and uses it to guide action generation. The representation is therefore encouraged to capture **what is changing and where the task is heading**, while execution-specific spatial detail can remain available to the Action Expert. In this way, joint-embedding prediction serves as a mechanism for organizing representations for control, rather than as an additional generative objective.

Contributions.

- **Policy-learned predictive target space.** We formulate latent world modeling such that the **predictive target space is learned jointly with the policy**, rather than prescribed by a pretrained encoder or a staged representation. This lets predictive learning and action supervision

jointly organize the representation used for control.

- **Asymmetric prediction–control interface.** We design the Future Expert to operate only on the compact predictive representation, while the Action Expert additionally accesses current-frame features. This allows the **predictive target space to focus on task-relevant evolution** without being forced to retain all execution-level spatial detail.
- **Empirical and representation-level evidence.** Across simulation and real-robot evaluations, Copper-Policy combines strong control performance with efficient training. Ablations and representation diagnostics link robustness to common closed-loop shifts to the learned World representation and show the complementary role of current-frame features.

2. Related Work

A broad line of work augments robot policy learning with signals derived from future observations or prediction, spanning world action models, future-conditioned policies, and joint-embedding prediction. A key distinction is the target space, which determines what information the representation retains for policy. Our comparison concerns representations that the policy reads for action decoding. In Copper-Policy, the predictive target space is the same compact World representation read by the Action Expert.

Fixed target spaces preserve information beyond what control requires. Fast-WAM [46], LingBot-VA [25], and Cosmos-Policy [22] predict future visual targets in pretrained video-generation latents, whereas earlier video-prediction policies generate future images or videos directly [9, 12, 23]. Both substrates preserve information needed for visual synthesis and may retain appearance not relevant to action prediction. Other WAM targets include future RGB–D videos [16], evolving geometric relations [48], and RGB–depth–flow futures [50]; outside WAMs, the same pattern appears with targets that are not futures at all [24].

Control-oriented targets compress future information, but are typically pretrained or staged. A second line uses future observations to learn control-oriented policy states. FLARE [53] carries future tokens through action denoising and aligns them with the future-observation embedding of an action-aware vision-language encoder, pretrained in an earlier stage and kept as a stop-gradient EMA target encoder during policy training. WoG [39] first learns a compact future condition through action prediction, then freezes it while training the current-context pathway to predict that condition. Being-H0.7 [33] instead jointly aligns current latent queries with a future-informed posterior in-

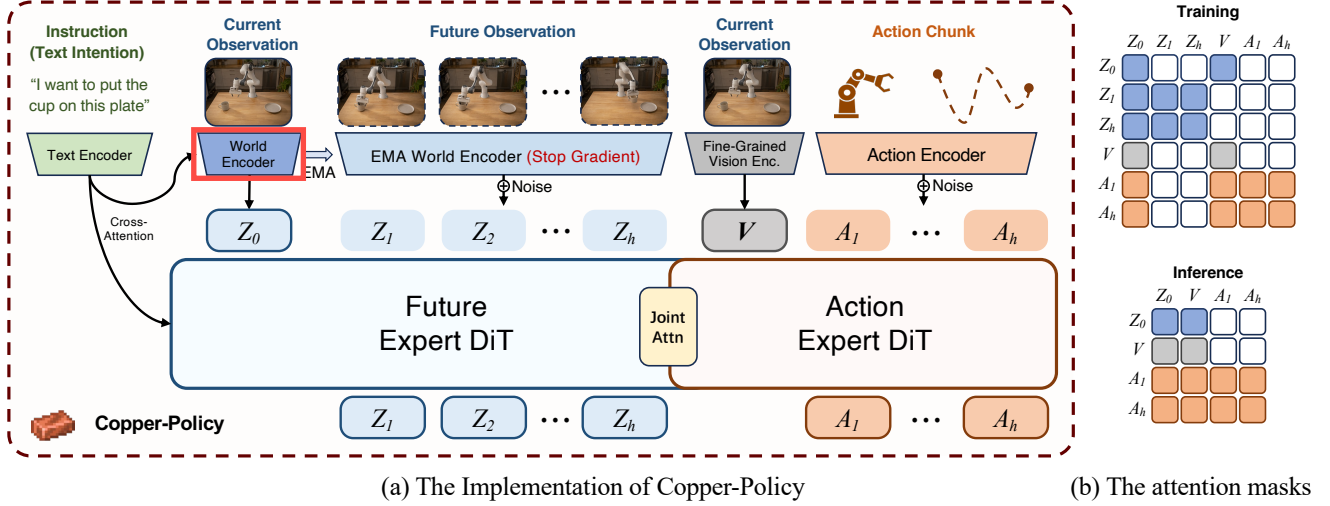


Figure 2. **Copper-Policy overview.** An online compactor produces the **World representation z_t** (blue), while a spatial encoder and grid sampler produce **current-frame tokens v_t** (gray). An EMA compactor supplies stop-gradient targets $\tilde{z}_{t+1:t+H_f}$. The Future Expert predicts these targets, and the Action Expert decodes the action chunk (orange) with access to the robot state q_t .

side the action model. Together, these methods show that privileged future information can shape compact, action-relevant policy states without test-time rollout. Our predictive target space is learned jointly with the policy. Current and future representations inhabit the same compact space, which action decoding also uses and shapes alongside prediction. Robot-relevant visual representations can also be pretrained on human and other Internet videos through time-contrastive [36], value-based [34], masked [38], or language-driven [20] objectives and then frozen. A separate line learns latent *actions* from unlabeled video [5, 45], encoding transitions rather than a compact observation representation.

JEPA predicts in representation space, but prior robot policies do not jointly shape that space through control. Joint-embedding predictive architectures (JEPAs) predict targets in representation space rather than reconstructing pixels. I-JEPA [1] and V-JEPA [2] use momentum-updated target encoders to define masked representation targets in images and videos, learning visual representations without an action objective and with deterministic predictors. VLA-JEPA [40] first pretrains a student on current observations to predict latent representations produced by a target encoder from future frames, then fine-tunes an action head. Copper-Policy instead learns the predictive target space jointly with action decoding and predicts in that same space by flow matching. Latent world models [17–19] put related predictive structures to a different use. DINO-WM [54] and LeWM [35] roll out frozen patch features and a learned latent, conditioned on candidate actions so that a planner can

	LIBERO	RoboTwin	Real
Camera views	2	3	3
Action space	delta EEf	abs. qpos	abs. qpos
Demos / task	50	50	300
Rollouts / task	50	100	50

Table 1. Per-benchmark data and rollout settings. The action space is either delta end-effector (EEF) pose or absolute joint positions (qpos).

search over them at test time.

Copper-Policy builds on Fast-WAM’s two-stream design and its finding that future prediction need not be used at deployment. The key difference is that the predictive target space is learned jointly with action decoding, so prediction shapes the same compact representation used for control.

3. Method

3.1. Overview

Figure 2 gives the architecture. At time t the policy observes o_t , a set of camera views, together with robot state q_t and task instruction s , and produces an action chunk of horizon H_a ,

$$a^+ := a_{t:t+H_a-1} \sim p(a^+ | o_t, q_t, s). \quad (1)$$

WAMs augment this with prediction targets over a future horizon H_f . Copper-Policy uses **temporal joint-embedding prediction** to learn a compact space conditioned on task intention and constrained by action decoding, implemented by an **online-EMA compactor pair** and two expert streams coupled through a structured attention mask. No future is predicted at test time.

Configuration	z_t	JE Pred.	v_t
Copper-Policy	✓	✓	✓
– w/o Joint-Embedding Prediction	✓	×	✓
– w/o Current-Frame Visual Features	✓	✓	×
– w/o World Input & Future Expert	×	×	✓
– w/o JE Pred. & Visual Features	✓	×	×

Table 2. Ablation configurations. *w/o World Input & Future Expert* gives the Action Expert only current-frame features and language.

3.2. Shared Representation with Online and EMA Compactors

Let f map an observation into a source space, either the features of a frozen pretrained encoder or the pixels themselves, and let $e_s = E_{\text{text}}(s)$ be a task-intention embedding from a text encoder, shared across the Future and Action Experts. The online compactor C_ϕ reads that source space under cross-attention to e_s and writes a compact target space:

$$z_t = C_\phi(f(o_t), e_s) \in \mathbb{R}^{K \times d}, \quad (2)$$

that is, K compact tokens of width d . The input layout and the compactor’s spatial mask determine K , yielding 28 tokens for LIBERO’s multi-view layout and 36 for the T-shaped RoboTwin and real-robot layout. Band-limited queries keep a coarse correspondence between compact tokens and image regions. Here f is a frozen V-JEPA-family encoder [2], while C_ϕ is trainable. Together, f and C_ϕ form the **World Encoder** of Fig. 2.

We regard o_t and $o_{t+\tau}$ from the same instructed trajectory as current and future views of one scene. Because the compactor reads e_s directly, the compact space can retain what the instructed task depends on while discarding the rest. We call z_t the **World representation**, a compact, task-conditioned behavioral prior assigned to the current view in a shared temporal embedding space. It serves as the *World input* to both expert streams in place of dense visual tokens.

Targets come from an **EMA copy $C_{\phi'}$ of the same compactor**, under stop-gradient:

$$\tilde{z}_{t+\tau} = \text{sg}(C_{\phi'}(f(o_{t+\tau}), e_s)), \quad \tau = 1, \dots, H_f. \quad (3)$$

Since a learned target space admits collapse, we follow BYOL [15] and subsequent joint-embedding practice [1]. **Gradients are stopped** through the target branch, and $C_{\phi'}$ tracks C_ϕ by EMA, so the target space co-evolves with the online representation. The frozen f is shared rather than copied. The future horizon matches the action horizon, $H_f = H_a$, and we construct N_f target nodes from this horizon. We write $\tilde{z}$ for the resulting target sequence and $z^{(u)}$ for its noised counterpart below.

3.3. Temporal Joint-Embedding Prediction

The Future Expert implements temporal joint-embedding prediction by predicting the compact future-view targets $\tilde{z}$

from z_t via **conditional flow matching** [28], a formulation also used by diffusion and flow policies [3, 11]. The prediction is conditioned on task intention e_s . We parameterize the flow from the clean target at $u = 0$ to pure noise at $u = 1$. For noise $\epsilon \sim \mathcal{N}(0, I)$, we interpolate

$$z^{(u)} = (1 - u) \tilde{z} + u \epsilon, \quad (4)$$

and train a velocity field $\pi_\theta(z^{(u)}, u \mid z_t, e_s)$, the Future Expert itself, to match $\epsilon - \tilde{z}$, the derivative of the interpolation path with respect to u . The **flow-matching Future Expert** is one of two streams in a joint self-attention transformer based on the mixture-of-transformers pattern [26, 46]. A structured mask lets it attend only to z_t and other future tokens, not directly to the current-frame tokens v_t or the robot state introduced next.

3.4. Action Expert and Current-Frame Vision

A **separate pathway** supplements z_t with current-frame spatial detail. A learnable spatial encoder g_ψ feeds a differentiable grid sampler S_ξ [14], a **visual bottleneck** that compresses a feature map into a few tokens, and together they produce $v_t = S_\xi(g_\psi(o_t))$. The Action Expert, the second stream of the joint self-attention transformer, additionally receives the robot state q_t as a single token projected alongside e_s , and decodes the action chunk with the same flow-matching formulation. For $a^{(u)} = (1 - u) a^+ + u \epsilon_a$ it predicts the velocity

$$\hat{v}^a = \pi_\theta^a(a^{(u)}, u \mid z_t, v_t, q_t, e_s), \quad (5)$$

trained to match $\epsilon_a - a^+$ and integrated at test time to give $\hat{a}^+$. Under the same mask, **action tokens attend to z_t , to v_t , and to other action tokens**, but never to the predicted future tokens. Thus, temporal joint-embedding prediction affects control only through the representations it shapes during training. Because the compactor is shared, the action loss directly constrains the space itself, **pressing the World representation to carry what the action decoder reads**.

3.5. Asymmetric Interface

This asymmetric interface keeps the predictive target compact while giving control direct access to current-frame detail. The structured mask of Secs. 3.3 and 3.4 gives the two objectives unequal readouts. Future queries attend to z_t and other future tokens, whereas action queries attend to z_t , the current-frame tokens v_t , and the state q_t . Within the current-step prefix, z_t and v_t attend to each other, so detail reaches future queries only after passing through the K compact tokens, while action decoding reads v_t directly. Consequently, joint-embedding prediction explains the future from z_t [1, 15], and its targets contain only compactor outputs. Action decoding can therefore use instantaneous

Method	Embodied Pretrain	WAM	LIBERO					LIBERO-Plus							
			Long	Goal	Object	Spatial	Avg.	Camera	Robot	Language	Light	Background	Noise	Layout	Total
Copper-Policy (ours)	×	✓ [‡]	95.2	95.6	99.4	98.8	97.25	78.67	73.74	67.34	95.27	84.20	94.44	76.52	80.85
$\pi_{0.5}$ [37]	✓	×	92.4	98.0	98.2	98.8	96.9	75.4	<u>77.5</u>	85.6	96.9	94.6	89.7	85.7	85.7
Cosmos-Policy [22]	×	✓	97.6	98.2	100.0	98.1	98.5	75.8	63.3	81.7	96.5	88.9	92.7	82.2	82.2
ABot-M0 [43]	✓	×	96.6	99.0	99.8	98.8	98.6	60.4	67.9	86.4	96.2	91.6	86.4	82.6	80.5
VLA-JEPA [40]	✓	✓ [‡]	95.8	97.2	99.6	96.2	97.2	64.2	67.7	88.1	91.8	93.4	65.8	<u>83.9</u>	77.9
X-VLA [52]	✓	×	97.6	97.8	98.6	<u>98.2</u>	98.1	23.4	89.7	75.7	88.2	96.0	62.7	71.8	71.4
Fast-WAM [46]	×	✓ [‡]	95.2	97.0	100.0	<u>98.2</u>	97.6	16.4	44.5	68.9	78.2	53.7	37.7	60.7	51.5
<i>Copper-Policy Variants</i>															
Copper-Policy			95.2	95.6	99.4	98.8	97.25	78.67	73.74	67.34	95.27	84.20	94.44	76.52	80.85
– w/o Joint-Embedding Prediction			91.2	95.0	100.0	97.4	<u>95.90</u>	76.17	78.58	<u>53.94</u>	93.52	90.33	<u>91.63</u>	<u>76.26</u>	<u>79.11</u>
– w/o Current-Frame Visual Features			90.6	95.4	99.0	96.8	<u>95.45</u>	76.49	71.23	52.77	95.01	74.26	81.32	72.72	74.11
– w/o World Input & Future Expert			90.8	92.4	98.8	97.2	94.80	66.79	67.23	52.18	95.18	88.20	87.07	74.49	74.56
– w/o Joint-Embedding Prediction & Visual Features			66.8	92.6	89.4	91.2	85.00	63.73	62.26	30.97	86.51	52.32	55.53	65.51	58.81

Table 3. LIBERO and LIBERO-Plus success rates (%). WAM marks explicit future modeling, and [‡] that no future is generated at deployment. Here and in later tables, bold and underlining mark the best and second best within a block, ties included.

detail without requiring z_t to duplicate it, while the action loss still shapes z_t toward information useful for control.

We test two predictions in Sec. 4.5. Joint-embedding prediction should reduce the overlap between perturbation-induced and task-driven temporal variation in z_t . The detail needed for precise action should live in v_t rather than in z_t , even though the action loss also presses on z_t .

3.6. Compact Token Interface

Each future target step adds a block of target tokens, so the target space sets the sequence length the experts attend over at every training step. In the real-robot setting Copper-Policy processes 244 tokens per sample against 392 for Fast-WAM, and the Future Expert attends over 180 of them against 360. Self-attention is quadratic in that length, so the size of the target space bounds training throughput as the prediction horizon grows.

3.7. Joint Objective and Inference

The temporal joint-embedding loss is

$$\mathcal{L}_{\text{pred}} = \mathbb{E} \left\| \pi_{\theta} \left(z^{(u)}, u \mid z_t, e_s \right) - (\epsilon - \tilde{z}) \right\|_2^2. \quad (6)$$

and the action loss regresses the same kind of velocity. We **optimize the action and prediction objectives jointly**; $C_{\phi'}$ never receives gradients.

Inference. At test time, the model receives only the current observation o_t , the robot state q_t , and the instruction s . The **online compactor and grid sampler produce z_t and v_t** ; the Future Expert stream runs once, as a context encoder, to populate its per-layer keys and values. The Action Expert then decodes $\hat{a}^+$ over 10 flow steps, attending jointly over that cache and its own tokens. No future tokens are instantiated.

4. Experiments

We evaluate Copper-Policy on LIBERO and LIBERO-Plus, RoboTwin, and three real-robot manipulation tasks, using the same ablation configurations throughout (Table 2).

4.1. Experimental Settings

LIBERO and LIBERO-Plus. Following the standard LIBERO protocol [29], we evaluate all 40 tasks across the Spatial, Object, Goal, and Long suites, using 50 rollouts per task and the same closed-loop replanning protocol for every variant. For LIBERO-Plus [13], we report task-weighted success over 10,030 perturbed tasks. Comparison numbers are from the respective papers or from the evaluation of Zhang et al. [49].

RoboTwin. We evaluate each variant on all 50 official tasks, with 100 rollouts per task in each setting. *Clean2Clean* trains and evaluates on the clean set, whereas *Clean2Rand* evaluates clean-trained policies under joint randomization of layout, distractors, background, and lighting. Comparison numbers are from the benchmark leaderboard [10] and the respective papers.

Real robot. We evaluate *Fold the Towel* for long-horizon deformable-object manipulation, *Put Banana* for pick-and-place under adversarial object placements, and *Stack Three Bowls* for multi-stage rigid-object manipulation. The platform comprises two 6-degree-of-freedom arms with standard grippers and three RGB cameras; no depth input is used. Actions are represented as 14-D absolute joint positions (qpos). We collect 300 teleoperated demonstrations per task at 30 Hz, totaling approximately 4.3 hours. An automatic scene generator produces 50 seeded initial configurations per task, on which each policy is evaluated using real-time chunking [4].

Method	Embodied Pretrain	WAM	RoboTwin		
			Clean	Random	Avg.
<i>Comparison with methods using embodied pretraining</i>					
$\pi_{0.5}$ [37]	✓	×	70.70	46.00	58.35
Spatial Forcing [24]	✓	×	77.20	26.74	51.97
X-WAM [16]	✓	✓	70.00	25.80	47.90
X-VLA [52]	✓	×	68.00	20.90	44.45
ABot-M0 [43]	✓	×	57.40	30.36	43.88
Copper-Policy (ours)	×	✓ [‡]	<u>70.84</u>	12.98	41.91 [†]
Xiaomi Robotics-0 [8]	✓	×	62.90	18.20	40.55
GalaxeaVLA [32]	✓	×	62.70	12.72	37.71
RDT-1B [30]	✓	×	34.50	13.72	24.11
<i>Methods without embodied pretraining</i>					
Copper-Policy (ours)	×	✓ [‡]	70.84	12.98	41.91
EventVLA [42]	×	×	65.60	15.70	40.65
Fast-WAM [46]	×	✓ [‡]	77.80	1.90	39.85
AHA-WAM [7]	×	✓	64.30	3.20	33.75
DP3 [47]	×	×	55.24	4.96	30.10
starVLA [44]	×	×	46.52	3.16	24.84
ACT [51]	×	×	29.74	1.74	15.74
<i>Copper-Policy Variants</i>					
Copper-Policy			70.84	12.98	41.91
– w/o Joint-Embedding Prediction			48.16	9.38	28.77
– w/o Current-Frame Visual Features			67.82	10.22	39.02
– w/o World Input & Future Expert			55.26	<u>10.22</u>	32.74
– w/o Joint-Embedding Prediction & Visual Features			51.66	9.42	30.54

Table 4. RoboTwin success rates (%). In the upper block, [†] marks the one entry trained without embodied pretraining; [‡] marks methods that generate no future at deployment.

4.2. Main Results

4.2.1. LIBERO and LIBERO-Plus

Copper-Policy averages 97.25% on standard LIBERO. Its 80.85% LIBERO-Plus total exceeds several embodied-pretrained VLA baselines; only Cosmos-Policy (82.2%), which fine-tunes Cosmos-Predict2 as its policy backbone, and large-scale embodied-pretrained $\pi_{0.5}$ (85.7%) score higher. Cosmos-Policy’s small overall lead is driven by Language, as Copper-Policy has the higher pooled success rate over the other six categories. It leads Camera and Noise, ranks behind only X-VLA and $\pi_{0.5}$ on Robot among comparison methods, and remains strong on Light.

4.2.2. RoboTwin

Among methods without embodied pretraining, Copper-Policy achieves the highest average success rate at 41.91%, ahead of EventVLA at 40.65% and Fast-WAM at 39.85%. Its clean-set score is on par with $\pi_{0.5}$ (70.84% versus 70.70%). Under randomization, however, it remains substantially below $\pi_{0.5}$ (12.98% versus 46.00%).

4.2.3. Real-Robot Evaluation

Each policy uses the protocol and initial-scene seeds described above. Copper-Policy obtains the highest average, 96.3 against 94.3 for $\pi_{0.5}$ and 84.0 for Fast-WAM, and is best on both rigid-object tasks (Fig. 3). Removing the World input and Future Expert drops the average from 96.3 to 84.7. Fast-WAM performs best on towel folding at 98

but reaches 60 on banana placement, where it often does not attempt the grasp.

4.3. Efficiency

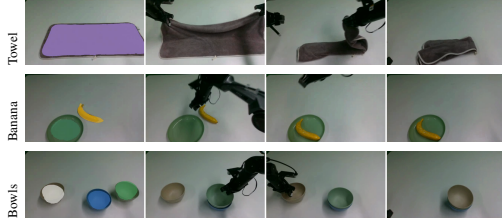
Predicting in a compact embedding space rather than generating video shortens the training sequence and permits smaller experts, putting the whole model at 2B trainable parameters. At a matched global batch of 128 on eight A100 GPUs, it uses less memory than $\pi_{0.5}$ and Fast-WAM and trains about $2\times$ and $6\times$ faster. The same configuration completes 30K steps in 9.67 hours on eight consumer RTX 5090 GPUs with plain data parallelism. End-to-end latency is about 85 ms, against 75 ms for $\pi_{0.5}$ and 80 ms for Fast-WAM. This slightly higher latency reflects the frozen 300M V-JEPA encoder in the World Encoder, which runs at every step.

Within Copper-Policy, optimizer sharding lowers memory from 31.83 to 24.53 GB while raising relative training time from $1.00\times$ to $1.08\times$; full sharding lowers memory to 21.24 GB but raises time to $1.50\times$ (Tab. 5). Copper-Policy fits on an RTX 5090 without sharding, whereas the two baselines use more memory despite full sharding. These values are operational cost references rather than a controlled systems benchmark because the methods differ in model size, sequence length, distributed strategy, and implementation.

4.4. Controlled Comparisons

The compact World representation supports robustness to common closed-loop shifts. Camera, Robot, Light, and Noise test changes that commonly arise during closed-loop execution. Copper-Policy leads on Camera and Noise, remains strong on Light, and ranks behind only X-VLA and $\pi_{0.5}$ on Robot. Removing the World input and Future Expert lowers overall success from 80.85% to 74.56%, with clear losses on Camera, Robot, and Noise. Retaining the World input without joint-embedding prediction recovers much of the overall score, but still trails the full model on Camera and Noise while scoring higher on Robot. These comparisons support the compact World representation’s value across common closed-loop shifts, while showing that predictive shaping benefits the shift categories differently.

Language, Background, and Layout expose the generalization gap. Language, Background, and Layout test broader instruction and scene generalization, where several embodied-pretrained methods lead. Language is the clearest gap at 67.34%. Copper-Policy uses frozen T5 without a pretrained VLM for language–image understanding or large-scale embodied pretraining, consistent with this result. More diverse training may help; we will explore the representations these categories need.



Configuration	Towel	Banana	Bowls	Avg.
$\pi_{0.5}$	95	94	94	94.3
Fast-WAM	98	60	<u>94</u>	84.0
Copper-Policy	93	96	100	96.3
w/o World Input & Future Expert	84	82	88	84.7
w/o Joint-Embedding Prediction	4	26	100	43.3
w/o Current-Frame Visual Features	2	46	14	20.7
w/o JE Pred. & Visual Features	0	4	0	1.3

Figure 3. **Real-robot evaluation.** The left panel shows four frames from one successful rollout per task, with target poses overlaid in the first column. The right panel reports scores that assign 1, 0.5, and 0 to full, half, and no success. $\pi_{0.5}$ uses embodied pretraining.

Method	Trainable params.	Training hardware	Parallelism	Peak mem. / GPU ↓	Relative training time ↓	Samples / s ↑	Est. time (30K steps) ↓	RTX 5090 inference ↓
Fast-WAM [46]	6B	8× A100 80GB	FSDP Fully Shard	62.42 GB	6.13×	14.36	74.3 h	~80 ms
$\pi_{0.5}$ [37]	3.3B	8× A100 80GB	FSDP Fully Shard	38.65 GB	1.96×	44.91	23.7 h	~ 75 ms
Copper-Policy (ours)	2B	8× A100 80GB	DDP	31.83 GB	<u>1.00×</u>	<u>88.0</u>	<u>12.1 h</u>	~85 ms
<i>Copper-Policy under alternative memory-sharding strategies (8× A100 80GB)</i>								
Copper-Policy	2B	8× A100 80GB	DDP	31.83 GB	<u>1.00×</u>	<u>88.0</u>	<u>12.1 h</u>	~85 ms
Copper-Policy	2B	8× A100 80GB	DDP + Optimizer Shard	<u>24.53 GB</u>	<u>1.08×</u>	81.8	13.0 h	~85 ms
Copper-Policy	2B	8× A100 80GB	FSDP Fully Shard	21.24 GB	1.50×	58.56	18.2 h	~85 ms
Copper-Policy (ours)	2B	8× RTX 5090 32GB	DDP	31.83 GB	0.80×	110.34	9.67 h	~85 ms

Table 5. Training cost at global batch 128, normalized to Copper-Policy under DDP on 8× A100 (1.454 s/step). Peak memory is reserved memory per GPU. FSDP Fully Shard covers ZeRO-3, FSDP, and FSDP2 full-shard; DDP + Optimizer Shard is ZeRO-1. The middle block changes only Copper-Policy’s sharding strategy. Estimated time assumes 30K steps; inference is end-to-end.

Cross-benchmark evidence for the representation design. The variant without joint-embedding prediction stays close to the full model on LIBERO-Plus but falls from 41.91 to 28.77 on RoboTwin and from 96.3 to 43.3 on the real robot. Together, these ablations and the four closed-loop shift categories support the compact World representation’s role in retaining task-relevant information across visual and robot changes. The larger losses on RoboTwin and the real robot suggest that this role matters more in physical control. Removing current-frame features lowers the LIBERO, LIBERO-Plus, and RoboTwin aggregate scores by at most 6.74 points, yet drops the real-robot average to 20.7, consistent with the need for current-frame detail in contact-rich execution (Sec. 4.5).

4.5. Representation Analysis

4.5.1. Temporal Structure and Channel Complementarity

Because the formulation is a claim about a representation, we inspect z_t directly. We feed frames from recorded training episodes through each variant offline and measure how z_t changes over time, treating the episode rather than the frame as the statistical unit so that long episodes do not dominate. The analysis covers 40 episodes per variant on LIBERO (ten tasks from each of the four suites), 18 on the real robot (six per task), and 20 on RoboTwin (five from

each of four tasks), all drawn with one fixed seed.

Joint-embedding prediction organizes the World representation in time. Table 6 reports every variant at $\Delta = 16$, and Fig. 4 traces cosine similarity across horizons. Removing joint-embedding prediction lowers cosine similarity in every domain, from 0.794 to 0.684 on LIBERO, from 0.902 to 0.756 on the real robot, and from 0.913 to 0.729 on RoboTwin, and raises displacement accordingly; without the joint-embedding objective the compactor still produces features, but they change faster over the episode. The paired comparison without v_t keeps the ordering (0.778 versus 0.665, 0.904 versus 0.773, and 0.890 versus 0.567). Across the complete attention set, joint-embedding prediction keeps z_t focused on manipulated objects and other task-relevant regions as an episode proceeds, whereas without the objective the maps more often retain maxima on background surfaces and lack a stable task-relevant centre.

The diagnostics accompany a behavioral reversal. *w/o Joint-Embedding Prediction* is worse than *w/o World Input & Future Expert* on RoboTwin (28.77% versus 32.74%) and on the real robot (43.3 versus 84.7), even though it has strictly more machinery. An unshaped World pathway supplies the Action Expert with a signal that drifts quickly and lacks a stable task-relevant focus, which is worse for control

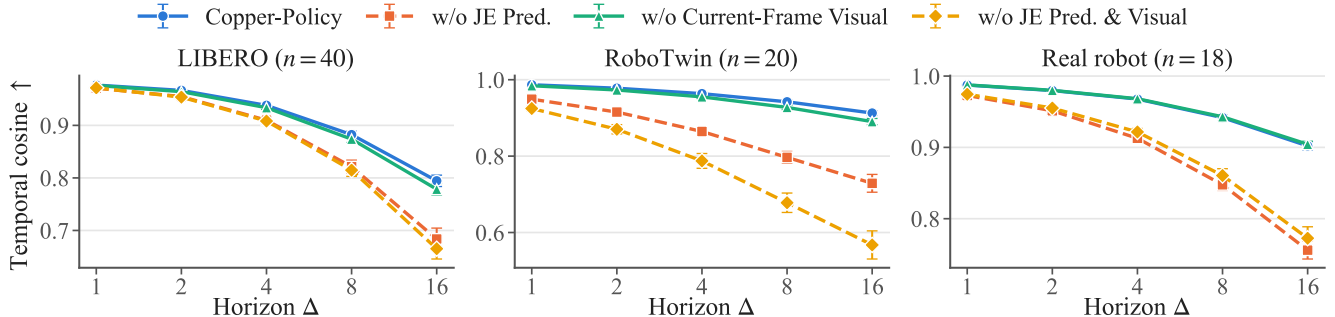


Figure 4. Cosine similarity of unit-normalized z_t across horizon Δ on LIBERO, RoboTwin, and the real robot, with 95% confidence intervals over episodes.

Configuration	LIBERO ($n = 40$)		RoboTwin ($n = 20$)		Real robot ($n = 18$)	
	cosine $\uparrow$	disp. $\downarrow$	cosine $\uparrow$	disp. $\downarrow$	cosine $\uparrow$	disp. $\downarrow$
Copper-Policy	0.794	0.634	0.913	0.413	0.902	0.434
– w/o Joint-Embedding Prediction	0.684	0.778	0.729	0.724	0.756	0.684
– w/o Current-Frame Visual Features	<u>0.778</u>	<u>0.660</u>	<u>0.890</u>	<u>0.463</u>	0.904	0.429
– w/o Joint-Embedding Prediction & Visual Features	0.665	0.809	0.567	0.921	0.773	0.658

Table 6. Temporal structure of the World representation, averaged over episodes. Cosine similarity and displacement are computed between unit-normalized representations 16 sampled frames apart; RoboTwin samples every fourth frame, so its separation spans more time and the three domains are not directly comparable. Bold and underlining mark the best and second best in each column.

than supplying no such signal at all. These measurements describe temporal smoothness of the representation; they are correlates of the mechanism, not proof of it.

The two pathways differ in temporal stability as well as spatial focus. Removing the current-frame pathway perturbs z_t far less than removing joint-embedding prediction does: cosine similarity moves from 0.794 to 0.778 on LIBERO and from 0.913 to 0.890 on RoboTwin, and on the real robot the two are indistinguishable (0.902 versus 0.904). Removing both joint-embedding prediction and the current-frame pathway is the largest drop on RoboTwin, though on the real robot it is no lower than removing joint-embedding prediction alone. Within the full model, z_t is also more stable than the pooled current-frame tokens v_t at the same separation, on LIBERO (0.794 versus 0.672), on the real robot (0.902 versus 0.827), and most clearly on RoboTwin (0.913 versus 0.245). The current-frame tokens therefore change faster over an episode than the World representation does.

4.5.2. Perturbation Geometry and Channel Probes

We draw 25 offline episodes from each of four RoboTwin tasks (*hanging mug*, *place empty cup*, *scan object*, and *stack three blocks*), take 16 uniformly spaced valid frames per episode for 1,600 clean states, and pair every state with five deterministic image perturbations, namely camera translation, illumination change, global colour cast, Gaus-

sian sensor noise, and blur, for 8,000 clean/perturbed pairs. All models run frozen in inference mode. Every split is by episode rather than by frame, and we report the mean and standard deviation over five deterministic episode-level splits. For a given rank k , we take U_N and U_T , the leading k directions of perturbation-induced and adjacent-frame temporal change, fit on training episodes alone, and report the overlap $\|U_N^\top U_T\|_F^2/k$ on held-out episodes, together with the energy of held-out perturbation change that falls in the temporal subspace. A random subspace of the same rank gives the baseline. Action probes use the same episode-level separation.

The first prediction holds. Joint-embedding prediction reorganizes rather than removes nuisance information. Perturbation identity remains linearly decodable from z_t (0.998 balanced accuracy), yet the perturbation-change and temporal-change subspaces overlap far less with the objective across ranks (Fig. 5a, top). For the leading- k perturbation-induced (U_N) and temporal-change (U_T) subspaces, overlap is $\|U_N^\top U_T\|_F^2/k$. At rank 8, it is 0.187 versus 0.798 without the objective and 0.021 at random, with the same ordering on all four tasks. Across paired clean and perturbed views, the shaped z_t also changes less in magnitude (0.134 versus 0.344 mean normalized L_2 distance), while its attention remains on the manipulated object (Fig. 5b–c). Perturbations thus act along directions less aligned with task evolution, the representation-level counterpart of the Noise and Camera gains above.

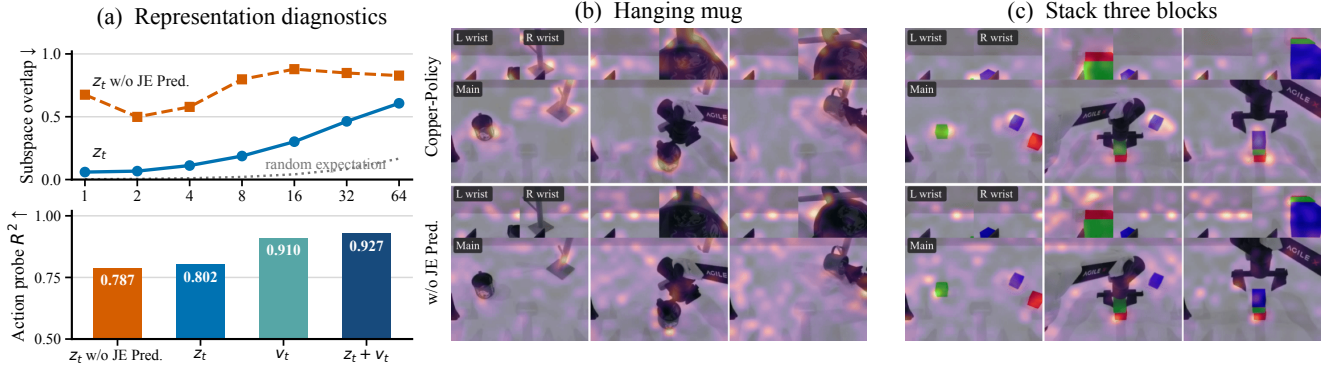


Figure 5. **Representation analysis.** (a) Perturbation–temporal subspace overlap across rank (top; dotted line: random expectation) and linear action-probe R^2 (bottom) on 100 RoboTwin episodes. (b–c) z_t attention across two RoboTwin episodes, comparing Copper-Policy (upper row) with *w/o Joint-Embedding Prediction* (lower row).

Representation	Geometry at rank 8		Paired perturbation		Action probe
	overlap ↓	pert. in temp. ↓	norm. L_2 ↓	cosine ↑	R^2 ↑
Copper z_t	0.187	0.140	0.134	0.990	0.802
– w/o Joint-Embedding Prediction	0.798	0.807	0.344	0.932	0.787
Current-frame tokens v_t	–	–	<u>0.171</u>	<u>0.969</u>	<u>0.910</u>
Dense V-JEPA features	–	–	0.428	0.904	0.831
Token-matched V-JEPA features	–	–	0.462	0.891	0.803
z_t and v_t together	–	–	–	–	0.927

Table 7. Representation diagnostics on 100 RoboTwin episodes. Geometry columns use train-only subspaces evaluated on held-out episodes; the random-subspace overlap is 0.021 and random energy is 0.021–0.023. Paired columns average 8,000 clean/perturbed pairs. Standard deviations over five episode-level splits are at most 0.006 for geometry and 0.021 for R^2 .

The second geometry measure tells the same story. At rank 8, only 0.140 of held-out perturbation-change energy lies in the temporal subspace of the learned z_t , compared with 0.807 without joint-embedding prediction (Tab. 7). This agrees with the low subspace overlap even though a probe can still identify the perturbation. The paired-view comparison adds a complementary measure: z_t has a cosine similarity of 0.990 between clean and perturbed images, compared with 0.932 without the objective. Together, these measures show that the objective changes both the direction and magnitude of the representation’s response to visual shifts.

The second prediction concerns the channels’ division of information. On held-out clean episodes, a linear action probe yields R^2 of 0.802 from z_t , 0.910 from v_t , 0.927 from both, and 0.787 from z_t without joint-embedding prediction (Fig. 5a, bottom). Instantaneous action detail is more directly accessible from v_t , and real-robot rollouts show why combining the channels matters. Without v_t , z_t is the only visual input to both experts and remains as persistent as in the full model (0.904 versus 0.902), yet the average task score falls from 96.3 to 20.7. Under our compact budget, the temporally shaped World channel cannot replace the current-frame pathway for precise control.

5. Conclusion

Copper-Policy learns a compact World representation jointly with its policy through joint-embedding prediction and action decoding. Its asymmetric interface preserves current-frame detail for precise control. Without embodied pretraining, it performs strongly across simulation and real-robot tasks, substantially improving on the two-stream Fast-WAM in LIBERO-Plus and real-robot scores while training faster with fewer parameters. Diagnostics show reduced overlap between perturbation-induced and task-driven variation in z_t and sustained attention to manipulated objects. Together with Camera, Robot, Light, and Noise results, this supports the representation’s value under common closed-loop shifts.

Limitations. Strong Camera, Light, and Noise results despite the larger Language gap suggest the World representation is not narrowly tied to visual appearance. Instruction grounding remains limited, while Background and Layout reveal further scene-generalization gaps. Future work will learn representations that better capture task intent and scene structure across these settings.

References

- [1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023. 3, 4
- [2] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024. 3, 4
- [3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 4
- [4] Kevin Black, Allen Z Ren, Michael Equi, and Sergey Levine. Training-time action conditioning for efficient real-time chunking. *arXiv preprint arXiv:2512.05964*, 2025. 5, 15
- [5] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Maria Elisabeth Bechtle, Feryal Behbahani, Stephanie C.Y. Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando De Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments. In *Proceedings of the 41st International Conference on Machine Learning*, pages 4603–4623. PMLR, 2024. 3
- [6] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Khalil Meftah, Maxime Ellerbach, Jess Moss, and Thomas Wolf. Lerobot: State-of-the-art machine learning for real-world robotics in pytorch. <https://github.com/huggingface/lerobot>, 2024. 15
- [7] Jisong Cai, Long Ling, Shiwei Chu, Zhongshan Liu, Jiayue Kang, Zhixuan Liang, Wenjie Xu, Yanan Mao, Weinan Zhang, Xiaokang Yang, et al. Aha-wam: Asynchronous horizon-adaptive world-action modeling with observation-guided context routing. *arXiv preprint arXiv:2606.09811*, 2026. 6
- [8] Rui Cai, Jun Guo, Xinze He, Piaopiao Jin, Jie Li, Bingxuan Lin, Futeng Liu, Wei Liu, Fei Ma, Kun Ma, et al. Xiaomirobotics-0: An open-sourced vision-language-action model with real-time execution. *arXiv preprint arXiv:2602.12684*, 2026. 6
- [9] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, et al. Gr-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024. 2
- [10] Tianxing Chen, Zhanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025. 5
- [11] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin CM Burchfiel, and Shuran Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, 2023. 4
- [12] Yilun Du, Sherry Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Josh Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. In *Advances in Neural Information Processing Systems*, pages 9156–9172. Curran Associates, Inc., 2023. 2
- [13] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025. 5
- [14] Yixu Feng, Zinan Zhao, Yanxiang Ma, Chenghao Xia, Chengbin Du, Yunke Wang, and Chang Xu. See what matters: Differentiable grid sample pruning for generalizable vision-language-action model. In *Forty-third International Conference on Machine Learning*, 2026. 4
- [15] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al. Bootstrap your own latent—a new approach to self-supervised learning. *Advances in neural information processing systems*, 33:21271–21284, 2020. 4
- [16] Jun Guo, Qiwei Li, Peiyan Li, Zilong Chen, Nan Sun, Yifei Su, Heyun Wang, Yuan Zhang, Xinghang Li, and Huaping Liu. Unified 4d world action modeling from video priors with asynchronous denoising. *arXiv preprint arXiv:2604.26694*, 2026. 2, 6
- [17] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems 31*, pages 2451–2463. Curran Associates, Inc., 2018. <https://worldmodels.github.io>. 3
- [18] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pages 2555–2565. PMLR, 2019.
- [19] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025. 3
- [20] Siddharth Karamcheti, Suraj Nair, Annie S Chen, Thomas Kollar, Chelsea Finn, Dorsa Sadigh, and Percy Liang. Language-Driven Representation Learning for Robotics. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, 2023. 3
- [21] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey

- Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In *Proceedings of The 8th Conference on Robot Learning*, pages 2679–2713. PMLR, 2025. 2
- [22] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, et al. Cosmos policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026. 2, 5
- [23] Po-Chen Ko, Jiayuan Mao, Yilun Du, Shao-Hua Sun, and Joshua B Tenenbaum. Learning to act from actionless videos through dense correspondences. In *International Conference on Learning Representations*, pages 40938–40958, 2024. 2
- [24] Fuhao Li, Wenxuan Song, Han Zhao, Jingbo Wang, Pengxiang Ding, Donglin Wang, Long Zeng, and Haoang Li. Spatial forcing: Implicit spatial representation alignment for vision-language-action model. *arXiv preprint arXiv:2510.12276*, 2025. 2, 6
- [25] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Fei Han, Mingrui Yu, Zelin Gao, Nan Xue, Xing Zhu, et al. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026. 2
- [26] Weixin Liang, LILI YU, Liang Luo, Srinu Iyer, Ning Dong, Chunting Zhou, Gargi Ghosh, Mike Lewis, Wen tau Yih, Luke Zettlemoyer, and Xi Victoria Lin. Mixture-of-transformers: A sparse and scalable architecture for multi-modal foundation models. *Transactions on Machine Learning Research*, 2025. 4
- [27] Yihan Lin, Jiawei He, Shifeng Bao, Chen Zhao, Yang Li, Xiaobo Wang, Yan Wang, Cheng Chi, and Jing Zhang. Jepa-wam: Learning vision-language-action policies with joint-embedding world modeling, 2026. 2
- [28] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2023. 4
- [29] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, pages 44776–44791. Curran Associates, Inc., 2023. 5
- [30] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations*, pages 29982–30009, 2025. 6
- [31] Xiao Liu, Yuguang Yang, Xi Wang, Kai Jiang, Cheng Chi, Yong Xu, Wenchao Ding, Yilun Chen, and Yan Wang. Stage-wam: Joint-embedding stage prediction for world-action models in robot manipulation, 2026. 2
- [32] Yicheng Liu, Zibin Dong, Baijun Ye, Tianyuan Yuan, Tao Jiang, Anqi Yang, Shicheng Cao, Haonan Liu, Yue Sun, Zihan Guo, Xiao Liu, Dong Ke, Changxun Pan, Chenru Wu, Tailai Cheng, Xiaoshu Ren, Xinlei Zhang, Jianning Cui, Zijie Zhao, Haoyu Zhang, Kaiming Xu, Haodong Yang, Bowen Zhang, Jiahui Niu, Shaoting Zhu, Shiduo Zhang, and Hang Zhao. G0.5: One autoregressive stream for robot reasoning and action, 2026. 6
- [33] Hao Luo, Wanpeng Zhang, Yicheng Feng, Sipeng Zheng, Haiweng Xu, Chaoyi Xu, Ziheng Xi, Yuhui Fu, and Zongqing Lu. Being-H0.7: A latent world-action model from egocentric videos. *arXiv preprint arXiv:2605.00078*, 2026. 2
- [34] Yecheng Jason Ma, Shagun Sodhani, Dinesh Jayaraman, Osbert Bastani, Vikash Kumar, and Amy Zhang. VIP: Towards universal visual reward and representation via value-implicit pre-training. In *The Eleventh International Conference on Learning Representations*, 2023. 3
- [35] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026. 3
- [36] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3m: A universal visual representation for robot manipulation. In *Proceedings of The 6th Conference on Robot Learning*, pages 892–909. PMLR, 2023. 3
- [37] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 2, 5, 6, 7, 13, 15
- [38] Ilija Radosavovic, Tete Xiao, Stephen James, Pieter Abbeel, Jitendra Malik, and Trevor Darrell. Real-world robot learning with masked visual pre-training. In *Proceedings of The 6th Conference on Robot Learning*, pages 416–426. PMLR, 2023. 3
- [39] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. *arXiv preprint arXiv:2602.22010*, 2026. 2
- [40] Jingwen Sun, Wenyao Zhang, Zekun Qi, Shaojie Ren, Zezhi Liu, Hanxin Zhu, Guangzhong Sun, Xin Jin, and Zhibo Chen. Vla-jepa: Enhancing vision-language-action model with latent world model. In *European Conference on Computer Vision*, pages 478–497. Springer, 2026. 2, 3, 5
- [41] Fan Yang, Yuting Su, Xiaobo Wang, Yuncheng You, Fugui Fan, Yuting Wu, Minghui Wu, Chenxu Zhao, JiaHong Ning, and Peiguang Jing. Lila-wam: Lightweight latent reasoning world-action model for robotic manipulation, 2026. 2
- [42] Ganlin Yang, Zhangzheng Tu, Yuqiang Yang, Sitong Mao, Junyi Dong, Tianxing Chen, Jiaqi Peng, Jing Xiong, Jiafei Cao, Jifeng Dai, et al. Eventvla: Event-driven visual evidence memory for long-horizon vision-language-action policies. *arXiv preprint arXiv:2606.20092*, 2026. 6
- [43] Yandan Yang, Shuang Zeng, Tong Lin, Xinyuan Chang, Dekang Qi, Junjin Xiao, Haoyun Liu, Ronghan Chen, Yuzhi Chen, Dongjie Huo, et al. Abot-m0: Vla foundation model for robotic manipulation with action manifold learning. *arXiv preprint arXiv:2602.11236*, 2026. 2, 5, 6
- [44] Jinhui Ye, Ning Gao, Senqiao Yang, Jinliang Zheng, Zixuan Wang, Yuxin Chen, Pengguang Chen, Yilun Chen, Shu Liu, and Jiaya Jia. StarVLA- α : Reducing complexity in vision-

- language-action systems. *arXiv preprint arXiv:2604.11757*, 2026. 6
- [45] Seonghyeon Ye, Joel Jang, Byeongguk Jeon, Se June Joo, Jianwei Yang, Baolin Peng, Ajay Mandlekar, Reuben Tan, Yu-Wei Chao, Bill Yuchen Lin, Lars Liden, Kimin Lee, Jianfeng Gao, Luke Zettlemoyer, Dieter Fox, and Minjoon Seo. Latent action pretraining from videos. In *International Conference on Learning Representations*, pages 28213–28239, 2025. 3
 - [46] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-wam: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026. 2, 4, 5, 6, 7, 13, 15
 - [47] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024. 6
 - [48] Jianjun Zhang, Jian Zhu, Taiyi Su, Chong Ma, Zitai Huang, Yi Xu, and Hanli Wang. Learning 4d geometric priors for inference-efficient world action models. *arXiv preprint arXiv:2607.05468*, 2026. 2
 - [49] Zhanguang Zhang, Zhiyuan Li, Behnam Rahmati, Rui Heng Yang, Yintao Ma, Amir Rasouli, Sajjad Pakdamansavoji, Yangzheng Wu, Lingfeng Zhang, Tongtong Cao, et al. Do world action models generalize better than vlas? a robustness study. *arXiv preprint arXiv:2603.22078*, 2026. 5
 - [50] Haoyu Zhao, Xingyue Zhao, Siteng Huang, Xin Li, Deli Zhao, and Zhongyu Li. RynnwORLD-4d: 4d embodied world models for robotic manipulation. *arXiv preprint arXiv:2607.06559*, 2026. 2
 - [51] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, 2023. 6
 - [52] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations*, pages 60580–60606, 2026. 2, 5, 6
 - [53] Ruijie Zheng, Jing Wang, Scott Reed, Johan Bjorck, Yu Fang, Fengyuan Hu, Joel Jang, Kaushil Kundalia, Zongyu Lin, Loïc Magne, Avnish Narayan, You Liang Tan, Guanzhi Wang, Qi Wang, Jiannan Xiang, Yinzhen Xu, Seonghyeon Ye, Jan Kautz, Furong Huang, Yuke Zhu, and Linxi Fan. Flare: Robot learning with implicit world modeling. In *Proceedings of The 9th Conference on Robot Learning*, pages 3952–3971. PMLR, 2025. 2
 - [54] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 79115–79135. PMLR, 2025. 3

A. Token Composition

A.1. Transformer Sequence Composition

Method	Current vis./world	Future	Action	Total
$\pi_{0.5}$ [37]	768	0	32	800
Fast-WAM [46]	120	240	32	392
Copper-Policy	68	144	32	244

Table A1. Transformer sequence composition per sample. For RoboTwin and real-robot runs, $|v_t| = 32$ and the T-shaped visual layout yields $K = 36$, giving 68 current tokens; $N_f = 4$ future target nodes give 144. The multi-view LIBERO layout yields $K = 28$. Of the current tokens the Future Expert sees only the K World tokens, since the mask of Sec. 3.3 excludes v_t . Each method uses 32 action tokens.

B. Representation Analysis

B.1. Representation Diagnostics Protocols

All representation diagnostics are read-only evaluations of frozen checkpoints. We run the model in evaluation mode with gradients disabled, use the same synchronized observations and language conditioning as in policy inference, and treat an episode—not an individual frame—as the statistical unit. For the World representation, the exported token and channel dimensions are flattened before temporal metrics are computed. For the current-frame pathway, the exported representation is the mean of its spatial tokens. Episode-level averaging gives each episode equal weight.

Temporal stability. For each episode we export the online World representation z_t and, for the full model, the mean-pooled current-frame representation v_t . At horizons $\Delta \in \{1, 2, 4, 8, 16\}$ we compute cosine similarity and Euclidean displacement between unit-normalized representations,

$$c_\Delta = \frac{1}{T - \Delta} \sum_t \hat{r}_t^\top \hat{r}_{t+\Delta}, \quad d_\Delta = \frac{1}{T - \Delta} \sum_t \|\hat{r}_{t+\Delta} - \hat{r}_t\|_2.$$

We average the resulting episode-level values and report 95% confidence intervals across episodes. The main temporal comparison uses 40 LIBERO episodes, 20 RoboTwin episodes, and 18 real-robot episodes. RoboTwin is subsampled every fourth frame, so the same index separation spans a longer physical interval there; its absolute values are therefore not compared numerically with LIBERO or the real robot.

Perturbation geometry and information probes. The larger perturbation analysis uses 100 RoboTwin episodes: 25 episodes from each of four tasks, with 16 uniformly spaced valid frames per episode. Each clean frame is paired with five deterministic visual perturbations—camera translation, illumination change, global color cast, Gaussian sensor noise, and blur—yielding 1,600 clean states and 8,000 paired examples. Splits are made by episode, never by frame, and all reported numbers are means over five deterministic episode-level splits. For geometry at rank $k = 8$, we fit the leading perturbation-change subspace U_N and adjacent-frame temporal-change subspace U_T on training episodes and evaluate

$$\frac{\|U_N^\top U_T\|_F^2}{k} \quad \text{and} \quad \frac{\|U_T U_T^\top \delta_N\|_2^2}{\|\delta_N\|_2^2}$$

on held-out episodes. A random subspace of the same rank is the reference. Linear nuisance and action probes are fit on the training episodes only; the former predicts perturbation identity, while the latter predicts the action chunk. For v_t , we mean-pool current-frame tokens before probing. We also report the paired clean/perturbed L_2 distance and cosine similarity.

Qualitative protocol. For the project-page z_t visualizations, the full model and *w/o Joint-Embedding Prediction* receive identical frames, task instructions, camera ordering, and rendering schedules. We render six evenly spaced steps per selected episode and preserve the native multi-view layout.

B.2. Attention-Map Generation

For the z_t attention shown in the main text and on the project page, we capture the attention probabilities in the final task-conditioned compactor read block. If A_{hqp} denotes the softmax probability from compact query q and head h to input patch p , the scalar patch score is

$$m_p^Z = \frac{1}{HQ} \sum_{h=1}^H \sum_{q=1}^Q A_{hqp}.$$

The patch sequence is then split according to the original camera grids, $m_p^Z \mapsto m^{Z,(m)} \in \mathbb{R}^{h_m \times w_m}$. The resulting z_t attention is the final-compactor read attention averaged over heads and compact queries.

For the current-frame representation map, we compute a post-hoc source-patch response from the tokens produced by the coordinate-based bilinear sampler. Let q_ℓ be a current-frame token and k_p the projected source-patch feature used by the sampler. With temperature $\tau = 0.10$, we compute

$$a_{\ell p} = \text{softmax}_p \left(\frac{\text{norm}(q_\ell)^\top \text{norm}(k_p)}{\tau} \right), \quad m_p^V = \frac{1}{L} \sum_{\ell=1}^L a_{\ell p}.$$

m^V is the *current-frame representation map*: it measures which source patches are most similar to the information carried by the sampled v_t tokens.

For rendering, each per-view map is reshaped to its patch grid, resized to the corresponding input image with bicubic interpolation, and optionally smoothed with a Gaussian kernel of $\sigma = 6$ pixels. The map is min-max normalized independently within each view and overlaid with opacity 0.45 on the min-max normalized RGB frame with opacity 0.55. We use the MAGMA colormap for m^Z and VIRIDIS for m^V ; color intensity is comparable within a map family and frame. Multi-camera panels preserve the model’s camera order; for three-view RoboTwin observations the display uses the T-shaped canvas with the two wrist views above the front view.

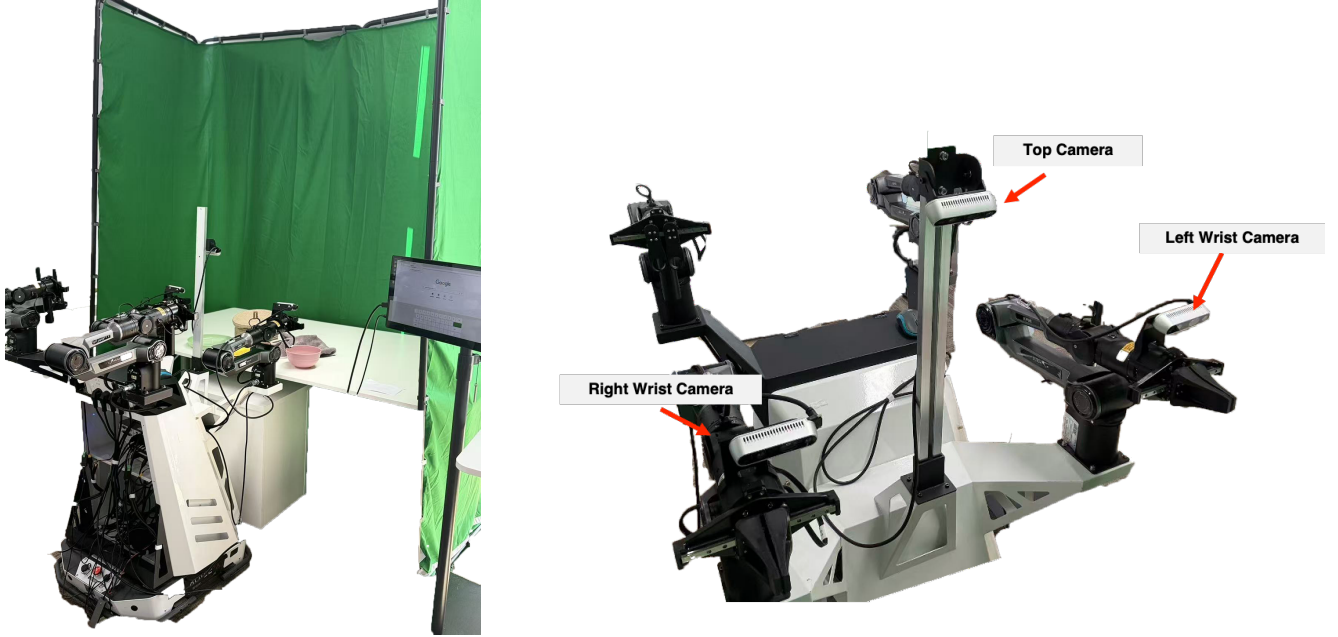


Figure A1. Real-robot experimental platform. The AgileX Cobot is equipped with two Piper arms and standard grippers (left); three Intel Corp. RealSense D435 cameras provide one top and two wrist views (right).

C. Real-Robot Evaluation Details

C.1. Experimental Platform

Our experiments use an AgileX Cobot equipped with two 6-degree-of-freedom Piper arms and standard grippers (Fig. A1). Robot communication, state acquisition, and joint-position command execution use the SDK provided by AgileX. Three

Intel Corp. RealSense D435 cameras provide multi-view observations, of which the policy uses only the RGB streams. The action space comprises 14-D absolute joint positions (qpos). Teleoperated demonstrations are stored in the LeRobot dataset format [6].

C.2. Real-Time Chunking

Evaluation scenes are produced by an automatic generator rather than placed by hand. We fit the distribution of object positions and orientations in the training set, sample from it with jitter, and hold the seeds fixed. Every policy therefore faces the same 50 initial scenes per task.

Copper-Policy uses real-time chunking (RTC) [4] during training and deployment. During training, we simulate RTC delay by conditioning the Action Expert on a committed action prefix whose length is sampled from 0 to 6 steps. For real-robot deployment, asynchronous RTC executes the action chunk $a^+ := a_{t:t+H_a-1}$ in closed loop: while the robot executes a committed prefix, the policy server infers the remaining part of the next chunk from the latest multi-view observation. After an initial chunk is committed, each replanning step preserves a committed prefix whose length is determined by the estimated end-to-end inference delay, while the Action Expert regenerates the remaining, uncommitted portion; RTC then replans at 30 Hz. This overlap between execution and inference maintains continuous action execution despite inference latency. The committed-prefix mechanism is specific to real-robot execution and is distinct from the action tokens the Action Expert otherwise generates during training and simulation evaluation.

C.3. Protocol and Outcome Counts

All policies, including $\pi_{0.5}$ and Fast-WAM, follow the same evaluation protocol and scene setup. They are trained or fine-tuned on the same 300 demonstrations per task, deployed on the same platform with the same cameras and action space, and evaluated on 50 trials per task from shared initial-scene seeds. Table A2 reports full-success, half-success, and failure counts, scored as 1, 0.5, and 0. Trial outcomes are assigned by manual annotation of the recorded rollouts. A trial is a full success when the task goal is reached, a half success when the goal is only partially reached, and a failure otherwise. Half success is awarded only on towel folding, when the fold is completed but a visible crease or corner remains. Banana placement and bowl stacking admit no partial credit. The reported average uses all 150 trials per policy variant.

Variant	Full / half / failure counts			Avg. score
	Fold Towel	Put Banana	Stack Bowls	
$\pi_{0.5}$ [37]	47/1/2	47/0/3	47/0/3	94.3
Fast-WAM [46]	49/0/1	30/0/20	47/0/3	84.0
Copper-Policy	44/5/1	48/0/2	50/0/0	96.3
w/o World Input & Future Expert	41/2/7	41/0/9	44/0/6	84.7
w/o Joint-Embedding Prediction	1/2/47	13/0/37	50/0/0	43.3
w/o Current-Frame Visual Features	1/0/49	23/0/27	7/0/43	20.7
w/o Joint-Embedding Prediction & Visual Features	0/0/50	2/0/48	0/0/50	1.3

Table A2. Per-task outcome counts behind the aggregate real-robot scores.

Additional qualitative observations. Successful Copper-Policy rollouts sometimes reach the task goals along trajectories visibly different from the teleoperated demonstrations. Fast-WAM often does not attempt the grasp in banana placement. In an additional unscored check outside the 150 reported trials, rotating the banana toward orientations present in the demonstrations recovered some grasp attempts.